WIP: The Traininator: Making Machine Learning Accessible to Elementary School Students

Gordon Stein*, Dun Na*, Atefeh Behboudi†, Tolulope Famaye‡, Cinamon Bailey‡, Golnaz Arastoopour Irgens†
*LIVE Learning Innovation Incubator, Vanderbilt University, Nashville, USA
{gordon.stein, dun.na}@vanderbilt.edu
†Department of Teaching and Learning, Vanderbilt University, Nashville, USA
{atefeh.behboudi, golnaz.arastoopour.irgens}@vanderbilt.edu
‡Department of Education and Human Development, Clemson University, Clemson, USA
{tfamaye, cinamob}@g.clemson.edu

2025 IEEE Frontiers in Education Conference (FIE) | 979-8-3315-0105-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/FIE63693.2025.11328481

***Abstract*—This work in progress innovative practice paper describes the design and implementation of the Traininator, a simplified machine learning training and testing interface. The Traininator reimagines Google Teachable Machine with a more developmentally appropriate interface for elementary school-aged students, while also presenting opportunities to discuss ethical implications like algorithmic bias. Originally designed for an educational game, it is now intended as a reusable application enabling image classifier creation in other projects.**

Students are presented with playful and developmentally appropriate interfaces to create image classifiers. They are guided through choosing classes, gathering data, applying data augmentation, training, and evaluating their model with test datasets. Models are usable in a Scratch-based programming interface, allowing for practical applications. Using a transfer learning approach, model training is quick and efficient. This tool is grounded in constructionist learning design principles, game-based learning pedagogies, and literature around child-computer interaction. Continued development will follow a participatory design-based research approach as children and teachers are included as co-designers during testing and evaluation processes.

User testing has been conducted in a lab setting with graduate students. Future assessment will use mixed-methods research to gather usability feedback and learning outcomes with students in grades 3-5.

***Index Terms*—Elementary school, computational thinking, computer-based instruction.**

I. Introduction

As artificial intelligence (AI) and machine learning (ML) technologies become increasingly integrated into everyday life, there is a growing demand to introduce these concepts to learners at younger ages, including those in elementary school [1]. Preparing the next generation not only to use but also to have foundational technical knowledge of AI systems is essential to fostering a more informed and responsible society. Moreover, to fully understand the human-centered effects of AI systems, young students must engage in ethical and social issues related to the deployment of such technologies.

One pressing ethical issue in AI is algorithmic bias, which can arise when machine learning models are trained on datasets that reflect existing societal prejudices. These biases can perpetuate inequality and lead to unjust outcomes, especially when used in applications such as facial recognition, hiring tools, or predictive policing [2]. Teaching young learners about bias in AI is crucial for developing critical thinking and empowering them to question and evaluate the fairness of automated systems.

Despite the recognized importance of AI and ethics education for children, there remains a significant gap in the availability of developmentally appropriate tools that make visible the complexities of machine learning and its ethical implications to younger students [3]. Most existing ML platforms are designed with older students or adults in mind and often lack the scaffolding needed to make concepts like training data, classification algorithms, and bias accessible to younger audiences.

To address this gap, we introduce the Traininator, an age-appropriate machine learning image classification tool designed specifically for elementary school students. The Traininator allows young learners to create, train, and test their own classification models using images they collect, while simultaneously making visible their training datasets for purposes of evaluating algorithmic bias.

II. Background

Our approach is informed by constructionist [4] learning design principles, in its project-based nature, and game-based learning pedagogies [5], in the interactive processes and goal-setting used when evaluating models and its original contextual framing. The Traininator represents a project-based learning experience where students develop their mental models of how a computer learns as they train and deploy their machine learning models. In the original curriculum, these models are used within a role-playing game [6], [7] combined with educational robots and block-based programming tools, which studies have shown facilitate young students' computational thinking and learning [8], [9].

The project's design and development process focuses on the main principles of child-computer interaction [10] and is based in part on five years of participatory design work with children from prior studies [11], [12] . The functional design of the Traininator is inspired by Google's Teachable Machine platform [13] and Williams' How to Train your Robot Curriculum [14]. Teachable Machine provides a web

interface for training and sharing machine learning models across multiple modalities, including images, audio, and pose data. No coding experience is required to train a model, and users can export their models for use in other applications. In Williams' work, students create Teachable Machine models, export the model, import the model into a Scratch block-based programming interface, and then program a Bluetooth connected robot to respond to the classification results. Another project focusing on simplifying Teachable Machine is "GenAI Teachable Machine" [15], which also simplifies the interface and includes the ability to trigger text, sounds, images, or videos based on the model's predicted class. The Traininator further simplifies this interface, redesigning it entirely for a younger audience.

III. Design

The Traininator demonstrates machine learning concepts, but hides some more advanced technical aspects. For example, in Teachable Machine, the number of epochs, batch size, and learning rate used for training are selectable by the user, but not presented in a way that a novice could comprehend these hyperparameters or change them with confidence. Students using the Traininator engage directly with datasets, data augmentation, and concepts regarding evaluation of classifier models, with the exposed options chosen to both prevent confusion and prevent a misconfiguration from preventing the student's model from working with no clear explanation. Teachable Machine presents users with both accuracy and loss per epoch, while the Traininator simplifies this to only the final accuracy, as the meaning of loss as different from opposite to accuracy would be out of scope for an elementary school student.

Similar to Teachable Machine, the Traininator uses transfer learning to quickly create useful classifiers, even on low-end hardware. The feature model selected for this use is MobileNetV2 [16], with an additional fully-connected combination of a 128-neuron dense layer using ReLU activation and a final classification layer using softmax activation with neurons corresponding to each class the model is expected to identify. Feature vectors output from MobileNet are paired with the desired output class to train the additional layers. This requires only a small model to be trained locally, preventing limited hardware available to students from being a barrier to experiencing machine learning. Furthermore, it reduces the amount of time required to train the model to seconds, and is significantly more energy efficient compared to training a full model from scratch [17].

A. Training a Traininator Model

The user interface of the Traininator guides students step-by-step through creating an image classification model. When first creating a model in the Traininator, students are prompted to name their model and list the classes it will recognize. At this step, it is required to list at least two classes so that a meaningful classifier may be trained.

The next step is assembling training data for the model. For each class listed in the previous step, the student is able to add or remove images. In the current iteration, the process used to obtain training and testing data is defined by the curriculum, with the Traininator accepting images added through a file picker or drag-and-drop. The training data interface is shown in Figure 1.

While curating training datasets, the Traininator introduces students to a useful machine learning technique, data augmentation. Given the students' limited time and attention spans, it is expected that the images they gather or create will be limited in scope, and this may have negative impacts on their classifiers' performance. To create more robust classifiers despite the limitations of the available training data, data augmentation can be used to expand the training set with images created by transforming the existing images into new instances for the training set [18]. In the Traininator, a set of "Model Booster" options are presented to the student, including two geometric transformations (rotation with scaling and horizontal mirroring) and one photometric transformation (color space transform). These are presented to the user with the simplified names "Rotate", "Reflect", and "Recolor", respectively. Selecting one of these options will augment the training set by applying the selected transform to every image in it, with a selection of transformed images presented as a preview with the text "Boosted Images!" overlaid. Examples of these transformations are shown in Figure 2.

B. Testing a Traininator Model

At the beginning of the evaluation step, students are asked to set a target accuracy for their model to achieve. They set this value using a slider ranging from 70% to 100%, labeled as "Easier to do" but "Not as good" to "Harder to do" but "Does a good job". This value is locked in for the remainder of the process. The modal dialog with this slider is shown in Figure 3.

A similar interface to the training data interface is used for creating a test set, with the same file picker and drag-and-drop options for adding new images. However, these images are not assigned classes immediately, and the Model Booster is not available, replaced with the "Model Matrix", displaying a confusion matrix between the classes the model is able to classify. Once the test set has been added, students click a "Test Model" button and their trained model classifies each image in the test set.

To provide a more interactive experience, labeling the training set is performed in the Traininator as part of the model evaluation process. Labels for the test set images are provided during evaluation, by the user. The interface for this labeling process is shown in Figure 4. For each instance in the test set, students are shown the result of the model's classification. Under the model classification result, the student is presented with two buttons where they can select if the model was correct or incorrect. After this reviewing each image, the student is presented with the same view for the test set as before, but now each image is marked as correctly or incorrectly classified,

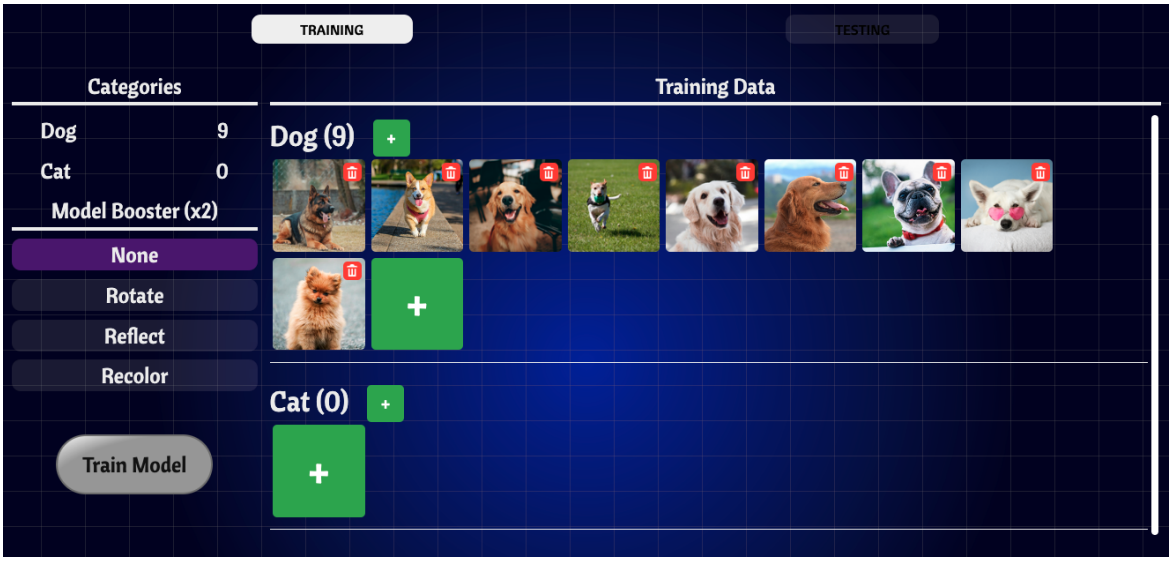

Fig. 1. Interface presented to a student user of the Traininator when adding training data. The "Dog" class has had images added to it, the "Cat" class has not. Training will be allowed when both classes are populated.

Fig. 2. Examples of "Model Booster" data augmentations. From top to bottom: rotation and scale, horizontal mirroring, color space transformation.

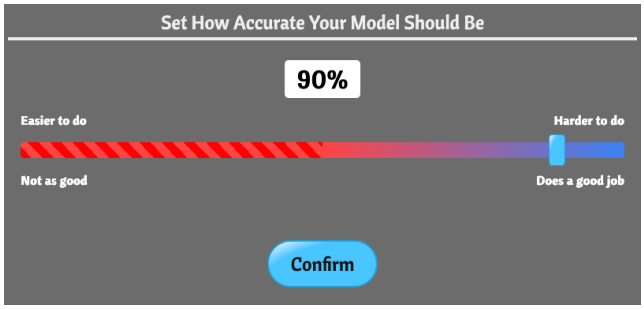

Fig. 3. Students are asked to set a threshold for acceptance of the trained model

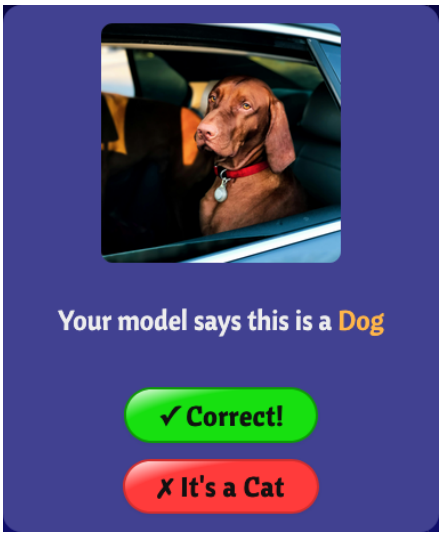

Fig. 4. Students evaluate test data by marking each classification as correct or incorrect.

the Model Matrix is populated with values, and the accuracy of the model is reported, as shown in Figure 5. This overall view, combined with the confusion matrix can help a student see potential biases in their model's inter-class accuracy. If a student makes a mistake during the reviewing process, there is a button on each image that allows the student to relabel the image.

With the model's accuracy evaluated, the student is given an option to save the model (if their stated threshold was met) or to return to the training step and retrain their model with additional training data. A successful model, if accepted, is saved to the student's account for later use. If a student chooses to retrain the model, the training data will remain as previously submitted, but open for modifications, including using a different Model Booster setting. However, the test set used previously will not be allowed to be reused, to avoid encouraging overfitting. While the students' subsequent test sets may not provide equivalent evaluations, this process is significantly more straightforward, and with instructor guidance it is unlikely that multiple iterations will be necessary for a successful classifier unless the student sets the threshold to the maximum.

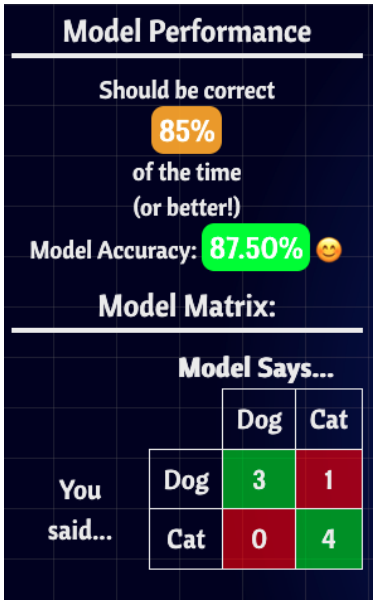

Fig. 5. After model is evaluated, students see the populated "Model Matrix" (confusion matrix) and the overall accuracy.

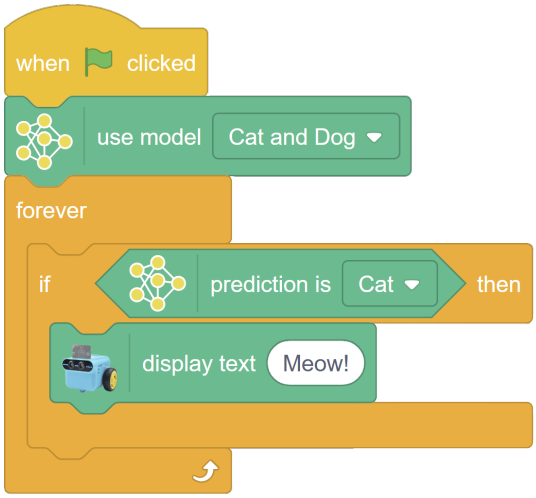

Fig. 6. Example program created with a Traininator model and Scratch plugin

C. Using a Traininator Model

To demonstrate the real-world usefulness of machine learning, the classifiers generated in the Traininator are accessible through a programming interface based on Scratch. A plugin derived from the Teachable Machine Scratch plugin developed for the DAILy curriculum [19] was created to provide an integration of classifiers into Scratch programs. The previous plugin requires students to copy a URL into block and run that block to load their model due to relying on Google's hosted version of Teachable Machine. Instead, our implementation is connected to the backend hosting the Traininator, allowing for a student's models to be provided as a drop-down list by name.

IV. Conclusion

Through the process of designing the Traininator, we build upon the strong foundation of existing tools for providing machine learning experience to K-12 students, while focusing on how these concepts can best be made both more accessible and more appealing to the youngest students. By providing specifically designed constraints and options, the Traininator makes visible the training and testing datasets and guide students toward investigating algorithmic bias. We hypothesize that the Traininator with its simplified and colorful interface, adjusted vocabulary, developmentally appropriate options, and careful guidance not only empowers elementary school students to create their own machine learning models but also makes it an activity they enjoy, technically understand, and engage with critically and ethically.

Continued development will focus on making the Traininator interface available as a package or platform, enabling its inclusion as a component in other curricula. The educational game it was originally developed for will begin to see deployment and testing in classrooms, allowing for student feedback and evaluation of the impact use of the Traininator has on elementary school students' opinions and outcomes regarding computer science and machine learning. This development process will utilize a participatory design model, in which students and teachers alike will contribute to the continued improvement of the tool. Additionally, future versions will include other forms of input and other model architectures, with similarly appropriate interfaces available when working with text, audio, or generative models.

Acknowledgment

This work was funded in part by the National Science Foundation (DRL-2448445) and Peabody College at Vanderbilt University. The opinions, findings, and conclusions do not reflect the views of the funding agencies, cooperating institutions, or other individuals.

References

[1] D. Touretzky, C. Gardner-McCune, and D. Seehorn, "Machine learning and the five big ideas in AI," *International Journal of Artificial Intelligence in Education*, vol. 33, no. 2, pp. 233–266, 2023.

[2] J. Buolamwini and T. Gebru, "Gender shades: Intersectional accuracy disparities in commercial gender classification," in *Conference on fairness, accountability and transparency*. PMLR, 2018, pp. 77–91.

[3] L. Morales-Navarro and Y. B. Kafai, "Unpacking approaches to learning and teaching machine learning in K-12 education: Transparency, ethics, and design activities," in *Proceedings of the 19th WiPSCE Conference on Primary and Secondary Computing Education Research*, ser. WiPSCE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3677619.3678117

[4] S. Papert, *Constructionism: A new opportunity for elementary science education*. Massachusetts Institute of Technology, Media Laboratory, Epistemology and ..., 1986.

[5] J. L. Plass, B. D. Homer, and C. K. K. and, "Foundations of game-based learning," *Educational Psychologist*, vol. 50, no. 4, pp. 258–283, 2015. [Online]. Available: https://www.tandfonline.com/doi/abs/10.1080/00461520.2015.1122533

[6] I. O. Adisa, I. Thompson, T. Famaye, D. Sistla, C. S. Bailey, K. Mulholland, A. Fecher, C. M. Lancaster, and G. Arastoopour Irgens, "S.P.O.T: a game-based application for fostering critical machine learning literacy among children," in *Proceedings of the 22nd Annual ACM Interaction Design and Children Conference*, ser. IDC '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 507–511. [Online]. Available: https://doi.org/10.1145/3585088.3593884

[7] G. Arastoopour Irgens, C. Bailey, T. Famaye, and A. Behboudi, "User experience testing and co-designing a digital game for broadening participation in computing with and for elementary school children," *International Journal of Child-Computer Interaction*, vol. 42, p. 100699, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2212868924000680

[8] Y. Zhang, R. Luo, Y. Zhu, and Y. Yin, "Educational robots improve K-12 students' computational thinking and STEM attitudes: systematic review," *Journal of Educational Computing Research*, vol. 59, no. 7, pp. 1450–1481, 2021.

[9] E. Roussou and M. Rangoussi, "On the use of robotics for the development of computational thinking in kindergarten: Educational intervention and evaluation," in *Robotics in Education*, M. Merdan, W. Lepuschitz, G. Koppensteiner, R. Balogh, and D. Obdržálek, Eds. Cham: Springer International Publishing, 2020, pp. 31–44.

[10] J. P. Hourcade, *Child-Computer Interaction*. Juan Pablo Hourcade, 2022.

[11] G. Arastoopour Irgens, H. Vega, I. Adisa, and C. Bailey, "Characterizing children's conceptual knowledge and computational practices in a critical machine learning educational program," *International Journal of Child-Computer Interaction*, vol. 34, p. 100541, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2212868922000599

[12] T. Famaye, G. A. Irgens, and I. A. and, "Shifting roles and slow research: children's roles in participatory co-design of critical machine learning activities and technologies," *Behaviour & Information Technology*, vol. 44, no. 5, pp. 912–933, 2025.

[13] M. Carney, B. Webster, I. Alvarado, K. Phillips, N. Howell, J. Griffith, J. Jongejan, A. Pitaru, and A. Chen, "Teachable Machine: Approachable web-based tool for exploring machine learning classification," in *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems*, ser. CHI EA '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1–8. [Online]. Available: https://doi.org/10.1145/3334480.3382839

[14] R. Williams, "How to train your robot: project-based ai and ethics education for middle school classrooms," in *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*, 2021, pp. 1382–1382.

[15] N. Pope, J. Kahila, H. Vartiainen, and M. Tedre, "Children's ai design platform for making and deploying ml-driven apps: Design, testing, and development," *IEEE Transactions on Learning Technologies*, 2025.

[16] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," 2019. [Online]. Available: https://arxiv.org/abs/1801.04381

[17] Y. Zhou, X. Lin, X. Zhang, M. Wang, G. Jiang, H. Lu, Y. Wu, K. Zhang, Z. Yang, K. Wang, Y. Sui, F. Jia, Z. Tang, Y. Zhao, H. Zhang, T. Yang, W. Chen, Y. Mao, Y. Li, D. Bao, Y. Li, H. Liao, T. Liu, J. Liu, J. Guo, X. Zhao, Y. WEI, H. Qian, Q. Liu, X. Wang, W. Kin, Chan, C. Li, Y. Li, S. Yang, J. Yan, C. Mou, S. Han, W. Jin, G. Zhang, and X. Zeng, "On the opportunities of green computing: A survey," 2023. [Online]. Available: https://arxiv.org/abs/2311.00447

[18] K. Alomar, H. I. Aysel, and X. Cai, "Data augmentation in classification and segmentation: A survey and new strategies," *Journal of Imaging*, vol. 9, no. 2, p. 46, 2023.

[19] I. Lee, S. Ali, H. Zhang, D. DiPaola, and C. Breazeal, "Developing middle school students' AI literacy," in *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*, ser. SIGCSE '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 191–197. [Online]. Available: https://doi.org/10.1145/3408877.3432513